placement.py — technical

`src/moe_pruning/model/placement.py · 133 lines · moe_pruning.model.placement`

Purpose

Single-device vs multi-GPU placement and device lookup helpers.

Step by step

Classes and functions in source order. Signatures, line ranges, raised exceptions and settings are extracted from the code; numbered steps describe what the code does.

`class ModelPlacement`

lines 31-43 · dataclass

Primary device plus optional multi-GPU device map and memory caps.

Field	Type	Default
`device`	torch.device	
`device_map`	str | dict | None	None
`max_memory`	dict | None	None

`ModelPlacement.sharded`

```
def sharded(self) -> bool
```

lines 37-38

True when the model is split over several devices.

`ModelPlacement.describe`

```
def describe(self) -> str
```

lines 40-43

Human-readable description for logs.

`parse_device_map`

```
def parse_device_map(value: str) -> str | dict | None
```

lines 46-60 · raises ValueError

Validate DEVICE_MAP (empty, a named strategy or a JSON object).

1. Empty -> None (single device). Starts with `{` -> JSON object (must be non-empty dict). Otherwise must be one of `auto`, `balanced`, `balanced_low_0`, `sequential`.

`parse_max_memory`

```
def parse_max_memory(value: str) -> dict | None
```

lines 63-74 · raises ValueError

`"0:13GiB,1:13GiB,cpu:20GiB"` -> `{0: "13GiB", 1: "13GiB", "cpu": "20GiB"}`.

1. Split on commas, each `key:amount`; numeric keys become GPU indices.

`placement_from_config`

```
def placement_from_config(cfg) -> ModelPlacement
```

lines 77-84 · settings DEVICE, DEVICE_MAP, MAX_MEMORY

Build the placement from DEVICE, DEVICE_MAP and MAX_MEMORY.

`from_pretrained_kwargs`

```
def from_pretrained_kwargs(placement: ModelPlacement) -> dict
```

lines 87-94

Keyword arguments for `from_pretrained` (weights go straight to their devices).

`place_model`

```
def place_model(model: nn.Module, placement: ModelPlacement) -> nn.Module
```

lines 97-116

Move an in-memory model to its device(s).

1. Not sharded: `model.to(device)`.
2. Named strategy: compute balanced per-device memory (except `sequential`) with accelerate's `get_balanced_memory` (respecting `MAX_MEMORY` and `_no_split_modules`), then `infer_auto_device_map`.
3. `dispatch_model(model, device_map)` moves weights and installs hooks that move activations between devices.

`module_device`

```
def module_device(module: nn.Module) -> torch.device
```

lines 119-124 · raises ValueError

Device of a module's first parameter (or buffer).

1. First non-meta parameter or buffer's device; error if none.

`input_device`

```
def input_device(model: nn.Module) -> torch.device
```

lines 127-129

Device that model inputs must be placed on (the input embedding's).

1. Device of `model.get_input_embeddings()` - where input tensors must be placed.

`model_devices`

```
def model_devices(model: nn.Module) -> list[torch.device]
```

lines 132-133

All devices holding parameters of the model.

Module constants

Name	Value
`NAMED_DEVICE_MAPS`	('auto', 'balanced', 'balanced_low_0', 'sequential')

Errors raised

Function	Exception	Message (f-string fields in braces)
`parse_device_map`	ValueError	DEVICE_MAP must be empty, one of {NAMED_DEVICE_MAPS} or a JSON object (got {value})
`parse_device_map`	ValueError	DEVICE_MAP JSON must be a non-empty object
`parse_device_map`	ValueError	DEVICE_MAP is not valid JSON: {exc}
`parse_max_memory`	ValueError	MAX_MEMORY entries must look like 0:13GiB (got {item})
`module_device`	ValueError	{type(module).__name__} has no materialised parameters